rustyweb

A Rust Primer

Claude

July 2026

Reading and understanding a web-archive server, one type at a time.

rustyweb - A Rust Primer

This document has two jobs at once:

1. Explain **how rustyweb works** as a program.
2. Use rustyweb's own source code to **teach Rust**, with a focus on the three things you asked about: **types**, **control flow**, and **error handling** (what other languages call "exceptions").

Every code sample below is real code from this repository, with a `file:line` reference so you can jump to it. Read it with the source open alongside.

A note before we start: Rust has **no exceptions**. There is no `try/catch`, no `throw`, no stack unwinding you're meant to catch in normal code. Instead, "something can fail" is encoded *in the type* a function returns. That single design decision shapes almost everything else, so we'll keep coming back to it.

---

1. The shape of the project

rustyweb is a **Cargo workspace** - a repo containing multiple related packages ("crates"). There are two:

```
crates/
  rustyweb-lib/    all the actual logic (a library crate)
     src/
         lib.rs          declares the modules
         collections.rs  the manifest of indexed WACZ files
         index.rs        the indexing pipeline
         warc.rs         low-level WARC record parsing
         wacz.rs         WACZ (ZIP) reading + CDX index
         search.rs       Tantivy full-text index wrapper
         pdf.rs          PDF text extraction
         server.rs       the Axum web server
  rustyweb-bin/    a thin command-line front-end (a binary crate)
      src/main.rs         argument parsing + dispatch
```

Why split it this way? Because a **library crate can be imported by tests and by the binary**, but a binary crate cannot be imported. Putting the logic in `rustyweb-lib` means the integration tests in `crates/rustyweb-lib/tests/` can call the real functions directly. The binary just parses arguments and calls the library.

Modules

`lib.rs` is tiny - it does nothing but declare which files are part of the library:

```rust
// crates/rustyweb-lib/src/lib.rs:1
pub mod collections;
```

```
pub mod index;
pub mod pdf;
pub mod search;
pub mod server;
pub mod warc;
pub mod wacz;
```

`mod collections;` means "there is a module named `collections`, find it in `collections.rs`." `pub` means it's visible outside the crate. Inside the code you then reach items with paths like `crate::collections::Collection` (the `crate::` root refers to this library) or, from the binary, `rustyweb_lib::collections::Collection`.

Visibility is opt-in. Everything is private by default; you write `pub` to export. You'll see this everywhere - `pub fn`, `pub struct`, `pub enum`. A function with no `pub` (like `resolve_sources` in `index.rs:66`) is an internal helper that cannot be called from outside its module.

---

2. Types: structs and enums

Rust's type system is where most of the "how does this work" story lives, because the types *are* the design. Let's look at the two kinds of custom type this codebase uses constantly.

Structs - "a bundle of named fields"

A `struct` groups related data. Here's the record type the WARC parser produces:

```
// crates/rustyweb-lib/src/warc.rs:6
#[derive(Debug, Clone)]
pub struct WarcRecord {
    pub record_id: String,
    pub concurrent_to: Option<String>,
    pub target_uri: String,
    pub timestamp: String,              // 14-digit: 20060102150405
    pub warc_type: String,
    pub http_status: Option<u16>,
    pub content_type: String,
    pub digest: String,
    pub payload: Vec<u8>,               // HTTP response body
    pub http_headers: Vec<(String, String)>,
    pub offset: u64,
    pub record_length: u64,
}
```

(That `#[derive(Debug, Clone)]` line is explained just below.)

Read the field types as a vocabulary lesson - they recur throughout the code:

Type	Meaning
`String`	An owned, growable, heap-allocated UTF-8 string.
`u16`, `u64`	Unsigned integers of a fixed width (16/64 bits). Rust makes you choose.
`Vec<u8>`	A growable array ("vector") of bytes. `Vec<T>` is the workhorse collection.
`Option<String>`	*Either* a string *or* nothing. This is how Rust models "maybe absent" - there is no `null`.
`Vec<(String, String)>`	A vector of 2-tuples - here, header name/value pairs.

The single most important one for a newcomer is **`Option<T>`**. Rust has no `null`. If a value might be missing, its type says so: `http_status: Option<u16>` means "there may or may not be an HTTP status." The compiler then *forces* you to handle the "missing" case before you can use the value - you cannot accidentally dereference a null. We'll see how you unwrap it in the control-flow section.

The `#[derive(...)]` attribute

```rust
#[derive(Debug, Clone)]
pub struct WarcRecord { ... }
```

`derive` auto-generates trait implementations so you don't hand-write boilerplate. The ones this codebase leans on:

- **`Debug`** - lets you print the value with `{:?}` for debugging.
- **`Clone`** - lets you make an explicit deep copy with `.clone()`.
- **`Default`** - gives a "zero value" via `T::default()` (see below).
- **`Serialize` / `Deserialize`** - from the `serde` library; lets the type convert to/from JSON. Look at `Collection` in `collections.rs:92`.
- **`PartialEq`** - lets you compare with `==` (used heavily in tests).

You'll see combinations like this on the manifest types:

```rust
// crates/rustyweb-lib/src/collections.rs:92
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Collection {
    pub id: String,
    #[serde(alias = "path")]
    pub source: Source,
    pub name: String,
    pub date_indexed: String,
```

```
    pub file_size: u64,
    pub sha256: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub description: Option<String>,
    ...
}
```

Those inner `#[serde(...)]` attributes fine-tune JSON handling: `skip_serializing_if = "Option::is_none"` means "if `description` is `None`, leave the field out of the JSON entirely" - so the manifest stays tidy. `alias = "path"` means "when reading JSON, also accept the old key name `path` for this field" - that's how the code stays backward-compatible with older manifests (see `collections.rs:96`).

Enums - "one of several shapes"

This is where Rust really differs from Python or Go. An `enum` is a type that is *exactly one of* a fixed set of variants, and **each variant can carry its own data**. The cleanest example in the codebase is `Source`:

```
// crates/rustyweb-lib/src/collections.rs:16
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
#[serde(from = "String", into = "String")]
pub enum Source {
    File(PathBuf),
    Url(String),
}
```

A `Source` is *either* a `File` holding a `PathBuf` (a filesystem path) *or* a `Url` holding a `String`. It cannot be both, and it cannot be neither. This models the domain precisely: a collection's WACZ lives either on local disk or at a remote URL, and those two cases need different handling.

The payoff is that you handle a `Source` with a **`match`**, and the compiler checks that you covered every variant:

```
// crates/rustyweb-lib/src/collections.rs:65
pub fn resolve(&self, home: &Path) -> Option<PathBuf> {
    match self {
        Source::File(p) if p.is_absolute() => Some(p.clone()),
        Source::File(p) => Some(home.join(p)),
        Source::Url(_) => None,
    }
}
```

Read this as: "given a `Source`, - if it's a `File` with an absolute path, return that path; - if it's any other `File` (i.e. relative), join it onto `home`; - if it's a `Url`, there is no local path, so return `None`."

The `if p.is_absolute()` on a match arm is a **guard** - an extra condition. The `_` in `Url(_)` means "a `Url`, but I don't care about the string inside." And the return type `Option<PathBuf>` again encodes "this might not produce a path" right in the signature.

There's another enum worth studying - `RawRecord` in the indexer - because it shows an enum used as a small tagged workflow value:

```
// crates/rustyweb-lib/src/index.rs:200
enum RawRecord {
    Html {
        url: String,
        timestamp: String,
        title: String,
        body: String,
    },
    Text {
        url: String,
        timestamp: String,
        text: String,
    },
}
```

Here each variant uses **named fields** (struct-like) rather than positional data. A WARC record contributes *either* an HTML page *or* a chunk of Browsertrix-rendered text, and the two carry different fields. Later the merge loop matches on it (`index.rs:254`) and does different things per variant. This is the classic Rust pattern: model the alternatives as an enum, then `match` to handle each.

The two enums you'll meet everywhere: `Option` and `Result`

`Option<T>` and `Result<T, E>` are just enums defined in the standard library. Understanding that they're ordinary enums demystifies them:

```
// (from the standard library, shown for reference)
enum Option<T> {
    Some(T),    // there is a value
    None,       // there isn't
}

enum Result<T, E> {
    Ok(T),      // success, carrying a T
    Err(E),     // failure, carrying an error E
}
```

That's it. `Option` is "value or nothing." `Result` is "success value or error value." Because they're enums, you handle them with `match`, `if let`, and the helper methods - which brings us to control flow and error handling, the heart of what

you asked about.

---

3. Error handling - Rust's answer to exceptions

This is the big one. In Python you'd write:

```python
try:
    data = read_file(path)          # might throw
except IOError as e:
    handle(e)
```

The failure is invisible in the function's signature - any call *might* throw, and you only find out by reading docs or crashing. Rust rejects this. A function that can fail **returns `Result<T, E>`**, and the caller cannot get at the success value without first acknowledging the error case.

`Result` in a signature

Look at almost any fallible function in this codebase:

```rust
// crates/rustyweb-lib/src/collections.rs:168
pub fn file_sha256(path: &Path) -> Result<String> { ... }
```

The return type `Result<String>` says "this either gives you a `String` or an error." (You'll notice it's written `Result<String>`, not `Result<String, E>` - that's because of `anyhow`, explained just below, which fixes the error type for you.) There is no way for a caller to use the `String` while ignoring that it might have failed - the type won't let them.

The `?` operator - the workhorse

Writing a `match` on every fallible call would be unbearable. The `?` operator is the ergonomic shortcut. Here's the full `file_sha256`:

```rust
// crates/rustyweb-lib/src/collections.rs:168
pub fn file_sha256(path: &Path) -> Result<String> {
    use sha2::Digest;
    use std::io::Read;

    let mut file = std::fs::File::open(path)?;       // <-- ?
    let mut hasher = sha2::Sha256::new();
    let mut buf = vec![0u8; 65536];
    loop {
        let n = file.read(&mut buf)?;                // <-- ?
        if n == 0 {
            break;
        }
```

```
        hasher.update(&buf[..n]);
    }
    Ok(bytes_to_hex(hasher.finalize().as_slice()))
}
```

`std::fs::File::open(path)` returns a `Result`. The `?` after it means:

> If this is `Ok(file)`, unwrap it and bind `file` to the value and carry on. If this is `Err(e)`, **stop this function immediately and return that error** to the caller.

So `?` is *early return on error*. It's the closest thing Rust has to exception-propagation, but it is explicit: you can see the `?` at every point where control might leave the function. Nothing is hidden.

Notice the last line: `Ok(bytes_to_hex(...))`. Because the function returns a `Result`, a successful result has to be *wrapped* in `Ok(...)`. And there's no semicolon - in Rust the last expression in a block is its value, so this line is the return value. (You could write `return Ok(...);` but idiomatic Rust omits the `return` and the semicolon on the final expression.)

`anyhow` - easy error propagation for applications

You keep seeing `use anyhow::Result;` at the top of files. `anyhow` is a library for application-level error handling. It gives you:

- **`anyhow::Result<T>`** - shorthand for `Result<T, anyhow::Error>`, where `anyhow::Error` is a catch-all error type that can wrap *any* underlying error. That's why the signatures say `Result<String>` with no second type parameter.
- **`?` across different error types** - `File::open` fails with an `io::Error`, `serde_json::from_str` fails with a `serde_json::Error`, yet both can be `?`-propagated in the same function because `anyhow::Error` absorbs either. In `CollectionManifest::open` you can see both in one function:

```
// crates/rustyweb-lib/src/collections.rs:127
pub fn open(index_dir: &Path) -> Result<Self> {
    let manifest_path = index_dir.join("collections.json");
    let collections = if manifest_path.exists() {
        let data = std::fs::read_to_string(&manifest_path)?;  // io::Error
        serde_json::from_str(&data)?                          // serde_json::Error
    } else {
        Vec::new()
    };
    Ok(Self { manifest_path, collections })
}
```

- **`.with_context(...)`** - attaches a human-readable message to an error as it propagates, so the final message is a breadcrumb trail rather than a

bare "file not found." This is used pervasively:

```
// crates/rustyweb-lib/src/index.rs:41
std::fs::create_dir_all(&index_dir)
    .with_context(|| format!("creating index dir {}", index_dir.display()))?;
```

If `create_dir_all` fails, the error becomes something like *"creating index dir /data/index: Permission denied (os error 13)"*. The `|| format!(...)` is a **closure** (an anonymous function) - it's only called if there actually is an error, so you don't pay to build the string on the happy path.

- **`anyhow!(...)`** and **`Context`** - `bail`-style construction of new errors. In the WARC reader:

```
// crates/rustyweb-lib/src/warc.rs:191
if n == 0 {
    return Err(anyhow!("unexpected EOF"));
}
```

`anyhow!("...")` builds an error from a message; wrapping it in `Err(...)` and returning it is how you *raise* an error in a `Result`-returning function.

Collecting many `Results` at once

A slick idiom appears repeatedly: turning an iterator of `Results` into a single `Result<Vec<_>>` that is `Ok` only if *every* element was `Ok`:

```
// crates/rustyweb-lib/src/index.rs:238
let warc_paths: Vec<_> = iter_warc_paths(wacz_path)?
    .collect::<Result<Vec<_>>>()
    .with_context(|| format!("listing WARC entries in {}", wacz_path.display()))?;
```

`iter_warc_paths` yields items of type `Result<String>`. Calling `.collect::<Result<Vec<_>>>()` says "gather these into a `Result<Vec<String>>`": if all items are `Ok`, you get `Ok(vec_of_strings)`; the moment one is `Err`, you get that `Err` and collecting stops. The trailing `?` then propagates it. This "collect a bunch of fallible things, fail if any fails" pattern is very common.

When failure *isn't* worth propagating: `Option` fallbacks

Not every "it didn't work" is an error worth bubbling up. Reading WACZ metadata is best-effort - a WACZ with no title isn't broken, it just needs a fallback. So the code uses `Option` and its combinators instead of `Result`:

```
// crates/rustyweb-lib/src/index.rs:117
let meta = read_datapackage(&local).unwrap_or_default();
let display_name = name
    .map(|n| n.to_string())
    .or_else(|| meta.title.clone().filter(|t| !t.trim().is_empty()))
    .unwrap_or_else(|| source_display_name(source));
```

Walk the fallback chain, which is a small essay in `Option` methods:

1. `name` is an `Option<&str>` (the `--name` CLI flag). `.map(|n| n.to_string())` turns `Some(&str)` into `Some(String)`, leaving `None` as `None`.
2. `.or_else(|| ...)` means "if still `None`, try this instead" - here, the WACZ title, but only if it isn't blank (`.filter(...)` drops it to `None` if the predicate fails).
3. `.unwrap_or_else(|| source_display_name(source))` means "if *still* `None`, compute a final fallback from the filename."

The result is a guaranteed `String` with a clean precedence order: explicit flag > WACZ title > filename. No exceptions, no nulls - just an `Option` narrowed step by step until a value is guaranteed.

`.unwrap_or_default()` on line 117 is the same idea: `read_datapackage` returns `Result<WaczMetadata>`, and if it errors we don't care - we just use a default (empty) `WaczMetadata`. That works because `WaczMetadata` derives `Default` (`wacz.rs:10`).

`panic!` - the escape hatch, and why it's rare here

Rust *does* have a way to abort: `panic!`. A panic unwinds the stack and (by default) crashes the thread. It's meant for **bugs and impossible states**, not for expected failures like a missing file. You'll see it mostly through two methods:

- **`.unwrap()`** - "give me the value inside this `Option`/`Result`, and panic if it's `None`/`Err`." Used when the programmer *knows* it can't fail.
- **`.expect("msg")`** - same, but with a custom panic message.

The interesting question for a learner is *when the authors judged a panic acceptable*. A representative example:

```
// crates/rustyweb-lib/src/search.rs:84
doc.add_text(schema.get_field(FIELD_DOC_TYPE).unwrap(), "page");
```

`get_field` returns an `Option` because a field name *might* not exist in the schema - but this code built the schema itself two functions over (`build_schema`, `search.rs:174`), so the field is guaranteed present. If it somehow weren't, that's a programming bug, and crashing loudly is the right response. The `.unwrap()` documents "this cannot fail unless I broke the schema."

Contrast that with a deliberately *descriptive* panic:

```
// crates/rustyweb-lib/src/search.rs:54
fn writer_mut(&mut self) -> &mut IndexWriter {
    self.writer
        .as_mut()
        .expect("SearchIndex opened read-only; no writer available")
}
```

The `SearchIndex` type holds an `Option<IndexWriter>` (`search.rs:23`): a writer is present when opened for indexing, absent when opened read-only for serving. Calling a write method on a read-only index is a logic error in *rustyweb's own code*, never something a user can trigger, so `expect` with a clear message is the right tool. This is a nice example of using `Option` in a field to encode a mode, then panicking only on genuine misuse.

Catching a panic on purpose: `catch_unwind`

There is exactly one place the code *does* treat a panic like a catchable exception, and the reason is instructive:

```
// crates/rustyweb-lib/src/pdf.rs:9
pub fn extract_pdf_text(bytes: &[u8]) -> Option<String> {
    let attempt = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| {
        pdf_extract::extract_text_from_mem(bytes)
    }));
    match attempt {
        Ok(Ok(text)) => { ... }   // extraction succeeded
        Ok(Err(_)) => None,       // clean parse error
        Err(_) => None,           // the library PANICKED; contain it
    }
}
```

The third-party `pdf-extract` crate can *panic* on malformed PDFs instead of returning an error. If that panic propagated, one bad PDF would abort the whole `rustyweb index` run. `catch_unwind` runs the risky call and converts a panic into an `Err`, which this function flattens into `None`. Note the *nested* `match`: the outer `Ok`/`Err` is "did it panic?", and the inner `Ok`/`Err` is "did extraction succeed?". This is the exception to the rule - and the comment in the source explains exactly why it's justified.

---

4. Control flow

Rust's control flow is expression-oriented: `if`, `match`, and blocks all *produce values*. Once that clicks, a lot of the code reads more naturally.

`match` - the centerpiece

You've seen `match` on enums already. Two things make it powerful: it's **exhaustive** (the compiler rejects your code if you forget a case) and it's an **expression** (it evaluates to a value). Here it computes a value that's then bound to a variable:

```
// crates/rustyweb-lib/src/wacz.rs:223
let status = match &obj.status {
```

```
    Some(serde_json::Value::Number(n)) => n.as_u64().unwrap_or(0) as u16,
    Some(serde_json::Value::String(s)) => s.parse().unwrap_or(0),
    _ => 0,
};
```

WACZ CDX files annoyingly store the HTTP status *sometimes* as a JSON number and *sometimes* as a quoted string. This `match` handles both shapes and falls back to `0`. The patterns dig two levels deep - `Some(Value::Number(n))` matches an `Option` that contains a JSON `Value` that is a `Number`, binding the inner value to `n` in one step. This nested **destructuring** is one of Rust's most pleasant features. The `_ =>` arm is the catch-all that makes the match exhaustive.

`if let` - "match just one pattern"

When you only care about *one* variant, a full `match` is overkill. `if let` is the shorthand:

```
// crates/rustyweb-lib/src/index.rs:344
if let Some(text) = crate::pdf::extract_pdf_text(&record.payload) {
    out.push(RawRecord::Html { ... });
} else {
    debug!(url = uri, "PDF text extraction yielded nothing; skipping");
}
```

Read `if let Some(text) = ...` as: "if the expression matches the pattern `Some(text)`, bind `text` and run the block." It's the standard way to say "do this only if the `Option` has a value." The optional `else` handles the `None` case.

`let ... else` - "bind or bail"

A newer, very readable form. When you need a value and there's no sensible way to continue without it, `let else` unwraps in the happy path and forces you to diverge (return/break/continue) otherwise:

```
// crates/rustyweb-lib/src/server.rs:397
let Some(c) = collections.iter().find(|c| c.id == id) else {
    return (StatusCode::NOT_FOUND, "collection not found").into_response();
};
// from here on, `c` is a plain &Collection - no nesting, no Option
```

Compare this to `if let`: with `if let` the bound variable only lives *inside* the block, which pushes your real logic into an indented branch. `let else` inverts that - `c` is available in the *rest of the function* at the outer indentation level, and the `else` block must not fall through (here it `return`s). This keeps the "not found" case as a quick guard and the main path flat and unindented. There's another example at `wacz.rs:133` and in `main.rs:44`.

Loops and iterators

Rust has ordinary loops (`loop`, `while`, `for`), and you'll see the imperative style where it fits - for example the WARC gzip reader is a hand-rolled `loop` with `break` because it's doing careful byte-offset bookkeeping (`warc.rs:64`).

But the *idiomatic* style for transforming collections is **iterator chains**. Here's the homepage building HTML cards from collections:

```
// crates/rustyweb-lib/src/server.rs:111
let cards: String = collections
    .iter()                          // borrow each Collection in turn
    .map(|c| {                       // transform each into an HTML string
        // ... build a card string from c ...
        format!(r#"<div class="card"> ... </div>"#)
    })
    .collect();                      // gather all the strings into one String
```

`.iter()` produces an iterator over borrowed elements; `.map(closure)` lazily transforms each; `.collect()` runs the whole chain and gathers the results. Note the target type annotation `let cards: String` - `.collect()` is polymorphic (it can build a `Vec`, a `String`, a `HashMap`, …), so it needs to know what you want. Here, collecting an iterator of `String`s into one big `String` concatenates them.

Iterators are lazy: nothing happens until a "consuming" method like `.collect()`, `.count()`, or a `for` loop drives them. You'll see rich chains like this one in the CDX parser:

```
// crates/rustyweb-lib/src/warc.rs:333
fn iso_to_14digit(s: &str) -> String {
    s.chars()                              // iterate over characters
        .filter(|c| c.is_ascii_digit()) // keep only digits
        .take(14)                           // at most 14 of them
        .collect()                          // build them back into a String
}
```

That turns `"2006-01-02T15:04:05Z"` into `"20060102150405"` in four composable steps, with no manual index arithmetic and no off-by-one risk.

Blocks as expressions

Because blocks yield values, you routinely assign the result of an `if` or a scoped block to a variable:

```
// crates/rustyweb-lib/src/server.rs:333
let table = if rows.is_empty() {
    String::new()
} else {
    format!("<table><tbody>{rows}</tbody></table>")
};
```

No ternary operator is needed - `if/else` *is* the expression. (Both branches must produce the same type, which the compiler enforces.)

A subtler use is a bare `{ ... }` block to **scope a lock** - we'll return to this in the concurrency section, but here's the shape:

```
// crates/rustyweb-lib/src/index.rs:276
let mut count = 0u64;
{
    let mut s = search.lock().unwrap();   // acquire lock
    for (url, m) in pages {
        s.index_page(...)?;
        count += 1;
    }
}   // <-- lock released here, at end of block
```

The lock is held only for the lifetime of `s`, and putting `s` inside an explicit block means the lock is released the instant the block ends. Control flow and resource management are the same thing in Rust, which is our next topic.

---

5. Ownership, borrowing, and lifetimes (the part that's new)

This is the concept with no direct analogue in Python/Java/Go, so it's worth a focused look even though you didn't name it - it underlies the types and control flow above.

The rule: every value has a single *owner*. When the owner goes out of scope, the value is dropped (freed). You can *borrow* a value instead of taking ownership, either immutably (`&T`, any number at once) or mutably (`&mut T`, exactly one at a time). The compiler checks all of this at compile time - there's no garbage collector.

Borrowing in signatures

Look at how functions take arguments:

```
// crates/rustyweb-lib/src/collections.rs:65
pub fn resolve(&self, home: &Path) -> Option<PathBuf> { ... }
```

- `&self` - "I borrow the `Source` immutably; I only read it." The caller keeps ownership.
- `home: &Path` - "I borrow a path; I won't take it or modify it."
- returns `Option<PathBuf>` - an *owned* `PathBuf` (note: `PathBuf` is owned, `&Path` is borrowed, exactly like `String` vs `&str`).

Compare with methods that need to mutate:

```rust
// crates/rustyweb-lib/src/collections.rs:141
pub fn upsert(&mut self, collection: Collection) {
    if let Some(pos) = self.collections.iter().position(|c| c.id == collection.id) {
        self.collections[pos] = collection;
    } else {
        self.collections.push(collection);
    }
}
```

- `&mut self` - "I need to *modify* the manifest," so this is a mutable borrow.
- `collection: Collection` (no `&`) - "I *take ownership* of this collection," because it's going to be stored inside `self.collections`. The caller can no longer use their `collection` variable after this call; it's been *moved* in.

This move-vs-borrow distinction is why you see `.clone()` and `.to_string()` sprinkled around: when the code needs its *own* copy to keep (because the original is borrowed or owned elsewhere), it explicitly clones. For example in `index_one`:

```rust
// crates/rustyweb-lib/src/index.rs:143
manifest.upsert(Collection {
    id,
    source: source.clone(),   // we only have &Source here, so clone to own it
    name: display_name,       // display_name is ours already, so just move it
    ...
});
```

`source` is a borrowed `&Source`, but the `Collection` needs to *own* its source, so `.clone()`. `display_name` is a `String` we built locally and don't need again, so it's moved in with no clone. Every clone in this codebase is a deliberate "I need my own copy here" - Rust never copies heap data implicitly.

The `_tmp` lifetime trick

Here's a beautiful, real example of ownership *as control flow*:

```rust
// crates/rustyweb-lib/src/index.rs:103
let (local, _tmp): (PathBuf, Option<tempfile::NamedTempFile>) = match source {
    Source::File(_) => (source.resolve(home).unwrap(), None),
    Source::Url(u) => {
        let tmp = download_to_temp(u)?;
        (tmp.path().to_path_buf(), Some(tmp))
    }
};
```

When indexing a remote URL, the code downloads it to a temp file. A `NamedTempFile` **deletes itself when it's dropped**. If the code just did `download_to_temp(u)?.path()`, the temp file would be dropped (and deleted!) at the end of that expression, before it could be indexed. So the file is bound to `_tmp` and kept alive for the whole function; only when `index_one` returns does

`_tmp` drop and the temp file get cleaned up. The leading underscore signals "I don't use this by name, I'm just holding it alive." Ownership *is* the cleanup mechanism - there's no `finally` block, the destructor runs automatically at the right moment.

You saw the same principle with the scoped lock block in §4, and again in `main.rs`:

```
// crates/rustyweb-bin/src/main.rs:139
let quiet = gag::Gag::stdout().ok();     // start suppressing stdout
let result = rustyweb_lib::index::index_location(location, &home, name.as_deref());
drop(quiet);                              // stop suppressing (explicit drop)
result?;
```

`gag::Gag` silences stdout while it's alive (to hide noisy PDF-library output); `drop(quiet)` ends the suppression at a precise point. Note also that `result` is computed *before* `drop`, and only `?`-propagated *after* - so that if indexing errored, stdout is restored before the error message prints.

---

6. Traits - shared behavior

A `trait` is like an interface: a set of methods a type can implement. Rust uses traits for everything from `==` to iteration to serialization. This codebase both *uses* standard traits and *implements* its own.

Implementing a standard trait

`Source` implements `Display` so it can be formatted with `{}`:

```
// crates/rustyweb-lib/src/collections.rs:86
impl std::fmt::Display for Source {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        f.write_str(&self.location())
    }
}
```

Once this exists, `format!("{source}")` and `println!("{source}")` work. The `impl Trait for Type { ... }` syntax is how you attach a trait's methods to your type.

`Source` also implements conversions via the `From` trait:

```
// crates/rustyweb-lib/src/collections.rs:74
impl From<String> for Source {
    fn from(s: String) -> Self {
        Source::parse(&s)
    }
}
```

```rust
impl From<Source> for String {
    fn from(s: Source) -> Self {
        s.location()
    }
}
```

This is what powers the `#[serde(from = "String", into = "String")]` attribute on the enum (`collections.rs:17`): serde serializes a `Source` by converting it *into* a `String` (just the path or URL), and deserializes by converting *from* a `String`. That's why the manifest JSON stores a source as a plain `"archive/x.wacz"` string instead of some tagged object - the two `From` impls define that mapping.

Implementing your own trait behavior for the standard library

`main.rs` has a more advanced example: a custom log formatter that colors WARN and ERROR lines. It implements the `tracing_subscriber` library's `FormatEvent` trait for a local wrapper type:

```rust
// crates/rustyweb-bin/src/main.rs:22
impl<S, N> FormatEvent<S, N> for ColorLineFormat
where
    S: tracing::Subscriber + for<'a> LookupSpan<'a>,
    N: for<'a> tracing_subscriber::fmt::FormatFields<'a> + 'static,
{
    fn format_event(&self, ctx: ..., mut writer: Writer<'_>, event: ...) -> std::fmt::Result
        ...
    }
}
```

Don't worry about the generics (`<S, N>`) and the `where` bounds on a first pass - the point is the *pattern*: to plug into the logging framework, you implement its trait. The `where` clause states requirements on the generic types ("`S` must be a `Subscriber` that also supports span lookup"). This is Rust's version of "implement this interface to hook into the framework."

Traits you implement to hook into machinery: `Read` / `BufRead`

The WARC parser needs a reader that also *counts bytes consumed* (to record file offsets). It defines a wrapper and implements the standard I/O traits on it:

```rust
// crates/rustyweb-lib/src/warc.rs:387
struct CountingBufReader<R: Read> {
    inner: BufReader<R>,
    count: u64,
}

impl<R: Read> Read for CountingBufReader<R> {
```

```
    fn read(&mut self, buf: &mut [u8]) -> std::io::Result<usize> {
        let n = self.inner.read(buf)?;
        self.count += n as u64;      // count the bytes as they flow through
        Ok(n)
    }
}
```

Because `CountingBufReader` implements `Read` and `BufRead`, it can be passed anywhere the standard library expects a reader, while transparently tracking position. `struct CountingBufReader<R: Read>` is a **generic struct**: it works for any inner reader type `R`, as long as `R` implements `Read` (that's the `<R: Read>` bound). This is how you write code that's reusable across many types without giving up compile-time checking.

Returning "some type that implements a trait": `impl Trait`

Several functions return `impl Iterator<...>` rather than a concrete type:

```
// crates/rustyweb-lib/src/warc.rs:28
pub fn iter_records(path: &Path) -> Result<impl Iterator<Item = Result<WarcRecord>>> {
    ...
    Ok(records.into_iter())
}
```

`impl Iterator<Item = Result<WarcRecord>>` means "I return *some* iterator that yields `Result<WarcRecord>` items - you don't need to know the exact concrete type." This hides implementation details and is very common for returning iterators and closures.

---

7. Concurrency - three flavors, all in this codebase

rustyweb touches three different concurrency tools, and they illustrate how Rust's ownership rules make concurrency safer.

7a. Data parallelism with Rayon

Indexing parses many WARC files, which is CPU-bound and embarrassingly parallel. Rayon turns a sequential iterator into a parallel one by changing `.iter()` to `.par_iter()`:

```
// crates/rustyweb-lib/src/index.rs:242
let per_warc: Vec<Vec<RawRecord>> = warc_paths
    .par_iter()                        // parallel iterator - runs across threads
    .map(|entry_name| {
        let tmp = extract_warc_from_wacz(wacz_path, entry_name)
            .with_context(|| ...)?;
        collect_page_records(tmp.path())
```

```
    })
    .collect::<Result<Vec<_>>>()?;    // still fails fast if any WARC errors
```

That one-word change (`iter` → `par_iter`) distributes the WARC parsing across CPU cores. It's safe because each closure works on its own `entry_name` and returns owned data - the compiler guarantees there's no shared mutable state being raced on. And note the error handling composes cleanly: the parallel work still collects into a `Result<Vec<_>>` and `?`-propagates the first failure.

7b. Shared mutable state with `Mutex`

The Tantivy search index, though, *is* shared mutable state - all those parallel tasks need to write into one index. That's what a `Mutex` (mutual exclusion lock) is for:

```
// crates/rustyweb-lib/src/index.rs:44
let search = Mutex::new(
    SearchIndex::open(index_dir.join("full_text").as_path())?
);
```

To touch the index, you `.lock()` it, which returns a guard giving exclusive access until the guard drops:

```
// crates/rustyweb-lib/src/index.rs:125
search.lock().unwrap().delete_collection(&id);
```

Here's the elegant part, tying back to §4 and §5: in `index_wacz` the writes are wrapped in an explicit block so the lock is held for the shortest possible time and released deterministically at the block's closing brace (`index.rs:276`). Rust's ownership model means you *cannot* access the data inside a `Mutex` without locking it - forgetting to lock isn't a runtime bug you might miss, it's a compile error. (`.lock()` returns a `Result` because a lock can be "poisoned" if a thread panicked while holding it; `.unwrap()` here says "if that happened, we're already in an unrecoverable state, so crash.")

At the end, `search.into_inner().unwrap()` (`index.rs:56`) consumes the `Mutex` and takes the `SearchIndex` back out - safe because by then all the parallel work has finished and there's a single owner again.

7c. Asynchronous I/O with Tokio

The web server is **async**. Network I/O spends most of its time waiting, so instead of one thread per connection, Tokio lets a small pool of threads juggle many connections by suspending tasks at their `.await` points.

You spot async code by two keywords: `async fn` to define, `.await` to call.

```
// crates/rustyweb-lib/src/server.rs:94
pub async fn serve(bind: &str, home: &Path) -> Result<()> {
    let app = router(home)?;
```

```
    let listener = tokio::net::TcpListener::bind(bind).await?;   // await
    tracing::info!("listening on {bind}");
    axum::serve(listener, ...).await?;                           // await
    Ok(())
}
```

An `async fn` doesn't run when you call it - it returns a *future*, a value representing "work that will produce a result later." `.await` is where you say "suspend here until this future is ready, letting other tasks run meanwhile." Note `.await?` combines the two ideas: await the future, then `?`-propagate the `Result` it produces.

The `#[tokio::main]` attribute on `main` (`main.rs:105`) sets up the async runtime so `main` itself can be `async`. And `tokio::select!` in the serve command (`main.rs:167`) runs three futures - the server, Ctrl-C, and SIGTERM - and proceeds as soon as *any one* completes, which is how graceful shutdown works:

```
// crates/rustyweb-bin/src/main.rs:167
tokio::select! {
    result = rustyweb_lib::server::serve(&bind, &home) => { result?; }
    _ = ctrl_c => {}
    _ = terminate => {}
}
```

Axum handlers, `State`, and `Arc`

Each route is an `async fn` handler. Shared state (the search index, the home dir) is wrapped in an `Arc` - an **A**tomically **R**eference-**C**ounted pointer that lets many tasks share ownership of one value safely:

```
// crates/rustyweb-lib/src/server.rs:43
let state = Arc::new(AppState { search, home: ..., index_dir: ... });
```

Handlers then receive it through Axum's `State` extractor:

```
// crates/rustyweb-lib/src/server.rs:108
async fn homepage(State(state): State<Arc<AppState>>) -> impl IntoResponse { ... }
```

The `State(state)` in the parameter position is **destructuring in a function argument** - it pulls the `Arc<AppState>` out of Axum's `State` wrapper and binds it to `state`. The return type `impl IntoResponse` (there's `impl Trait` again) means "I return something Axum knows how to turn into an HTTP response" - which, looking at the handlers, is often just a tuple `(StatusCode, headers, body)`. Axum provides the `IntoResponse` trait impls that make those tuples work.

---

8. Serde - types as the schema

You've seen `#[derive(Serialize, Deserialize)]` throughout. The philosophy worth absorbing: in Rust, **the struct definition IS the JSON schema**. You describe the shape you want as a type, and serde generates the parsing/writing code.

The WACZ metadata reader shows this vividly, including *local* structs defined right inside a function just to parse one thing:

```rust
// crates/rustyweb-lib/src/wacz.rs:43
#[derive(Deserialize, Default)]
struct Metadata {
    title: Option<String>,
    description: Option<String>,
    created: Option<String>,
    mtime: Option<i64>,
}
#[derive(Deserialize, Default)]
struct DataPackage {
    title: Option<String>,
    description: Option<String>,
    created: Option<String>,
    #[serde(default)]
    metadata: Option<Metadata>,
}
```

Every field is an `Option`, which is serde's way of saying "this JSON key might be absent - that's fine, you'll get `None`." Because these are declared *inside* `read_datapackage`, they're scoped to exactly where they're used and don't pollute the module. Parsing is then one line:

```rust
// crates/rustyweb-lib/src/wacz.rs:65
if let Ok(dp) = serde_json::from_str::<DataPackage>(&buf) {
    let nested = dp.metadata.unwrap_or_default();
    meta.title = clean(dp.title.or(nested.title));
    ...
}
```

`serde_json::from_str::<DataPackage>(&buf)` says "parse this text *as* a `DataPackage`." Note the `if let Ok(dp)` - parsing returns a `Result`, and here a parse failure is tolerated (the metadata is optional), so it's handled with `if let` rather than `?`. And `.or(...)` on `Option`s implements the "top-level value, else nested value" precedence in one expression.

For the CDX records that have inconsistent types, serde is told to keep the raw JSON value and the code coerces it manually - a good example of dropping to a lower level when the data is messy:

```rust
// crates/rustyweb-lib/src/wacz.rs:184
#[derive(Deserialize)]
struct CdxJson {
    url: Option<String>,
    status: Option<serde_json::Value>,   // could be number OR string
    offset: Option<serde_json::Value>,
    length: Option<serde_json::Value>,
    ...
}
```

`serde_json::Value` is "any JSON value, decoded later," used precisely because these fields aren't consistently typed in real WACZ files (`coerce_u64` at `wacz.rs:195` sorts them out).

---

9. Putting it together: one full trip through `index`

To see how the pieces connect, follow what happens when you run `rustyweb index archive/site.wacz`:

1. **`main`** (`main.rs:105`) is an `async fn` under `#[tokio::main]`. It initializes logging, then `Cli::parse()` (from `clap`) turns argv into the typed `Commands` enum. A `match cli.command` dispatches to the `Index` arm (`main.rs:121`).
2. That arm calls **`index_location`** (`index.rs:39`). It creates the index directory (`?`-propagating any I/O error with context), opens a `SearchIndex` wrapped in a `Mutex`, and calls `resolve_sources` to expand the argument into a `Vec<Source>` (a single file here, but it handles directories and URLs too).
3. For each source, **`index_one`** (`index.rs:93`) runs. It uses the `(local, _tmp)` ownership trick to get a readable local path (downloading if it's a URL). It reads metadata (`read_datapackage`, best-effort via `unwrap_or_default`), computes the display name through the `Option` fallback chain, deletes any prior documents for this collection (upsert semantics), and indexes the pages.
4. **`index_wacz`** (`index.rs:232`) lists the inner WARC files and parses them **in parallel with Rayon** (`par_iter`), producing `Vec<RawRecord>` per WARC. The `RawRecord` enum distinguishes HTML pages from rendered-text records.
5. Back in `index_wacz`, all records are **merged into one `MergedPage` per URL** using a `HashMap` and a `match` on the `RawRecord` enum (`index.rs:254`). Each merged page is written to Tantivy under a scoped `Mutex` lock.

6. Down in `warc.rs`, `iter_records` did the byte-level work: detecting gzip by magic bytes, decompressing member by member while tracking offsets with `CountingBufReader`, and parsing WARC headers with a hand-written `loop`. Every fallible step returns `Result` and uses `?` or collects into `Result<Vec<_>>`.
7. Finally `index_location` commits the Tantivy writer and saves the manifest (`manifest.save()`), both `?`-propagating errors up to `main`, whose `-> Result<()>` return type means any error bubbles all the way out and is printed by the runtime.

Notice there is **not a single `try/catch` in that whole chain**. Failure flows through `Result` and `?`; "maybe absent" flows through `Option`; the one place a panic could occur (a bad PDF) is explicitly contained. That is idiomatic Rust error handling end to end.

---

10. A cheat-sheet of the idioms in this codebase

You see…	It means…
`Option<T>`	Maybe a `T`, maybe nothing. No nulls.
`Result<T>`	Success (`T`) or failure. (`anyhow` fixes the error type.)
`foo()?`	Unwrap on success, else return the error from *this* function.
`.unwrap()` / `.expect("..")`	Assume success; **panic** if wrong. Used for "can't happen" cases.
`.with_context(|| ...)`	Attach a message to an error as it propagates.
`match x { ... }`	Exhaustively handle every case; also produces a value.
`if let Some(v) = x`	Do something only when `x` matches one pattern.
`let Some(v) = x else { return ... }`	Unwrap or bail; keeps the happy path unindented.
`.map / .filter / .collect`	Iterator pipeline; lazy until collected.
`&T` / `&mut T`	Borrow immutably / mutably. No ownership transfer.
`T` (owned, no `&`) as an argument	The function *takes ownership* (a move).
`.clone()`	Explicit deep copy, because Rust never copies heap data implicitly.
`impl Trait` (return position)	"Some concrete type implementing this trait."

You see…	It means…
`#[derive(...)]`	Auto-generate trait impls (Debug, Clone, Serialize, …).
`async fn` / `.await`	Asynchronous function / suspend point (server code).
`Arc<T>` / `Mutex<T>`	Shared ownership / mutual-exclusion lock (concurrency).
`.par_iter()`	Rayon: run this iterator across CPU cores.

---

11. Where to go next

If you want to solidify this, the highest-leverage things to read and tinker with, in order:

1. **The Rust Book**, chapters 4 (ownership), 6 (enums + `match`), 9 (error handling), and 13 (iterators + closures). Those four map almost exactly onto what this codebase leans on.
2. **`collections.rs`** is the best single file to study first - it's small and shows structs, an enum, trait impls (`From`, `Display`), serde, and `match` all in one place, with tests that double as usage examples (`collections.rs:196`).
3. **`pdf.rs`** is tiny and is the one place error handling gets subtle (`catch_unwind`) - a good "why is this here?" puzzle.
4. Try a small change and let the compiler teach you: remove a `?`, or change a `&Path` to `Path`, and read the error. Rust's compiler errors are unusually good tutors.
5. The `#[cfg(test)]` modules at the bottom of most files are runnable, readable specifications of what each function does. `cargo test` runs them all.

The tests are worth emphasizing: nearly every module ends with a `mod tests` block, and reading a function's tests alongside the function is often the fastest way to understand both the code and the Rust patterns it uses.

Part II - Deeper Dives

Part I gave you the working vocabulary. This part goes deeper on the two concepts that are genuinely unique to Rust and trip up most newcomers - **lifetimes** and **async** - and then tours the smaller Rust-specific features that appear throughout rustyweb. These two topics are connected: async forces lifetime questions into the open, which is why they belong together.

---

12. Lifetimes, in depth

The problem lifetimes solve

Recall the ownership rule from §5: when a value's owner goes out of scope, the value is freed. A **reference** (`&T`) borrows a value without owning it. That raises an obvious danger: what if the reference outlives the thing it points to? You'd have a *dangling pointer* - a reference to freed memory. In C that's a classic security bug. In Rust it's a **compile error**.

A **lifetime** is the compiler's name for "the span of code during which a reference is valid." Lifetimes aren't something you compute or that exist at runtime - they're labels the borrow checker uses to prove, at compile time, that no reference outlives its data. Most of the time you never write them, because the compiler infers them. But they're always there.

Lifetime elision - why you rarely see them

Look at this function from the binary:

```rust
// crates/rustyweb-bin/src/main.rs:249
/// First 8 characters of a hex hash for compact display.
fn short_hash(hash: &str) -> &str {
    hash.get(..8).unwrap_or(hash)
}
```

This returns a `&str` that is a **slice into the input** `hash` - no new string is allocated, the return value points *into* the caller's data. That's only safe if the returned reference doesn't outlive `hash`. The compiler enforces exactly that, because behind the scenes this signature means:

```rust
fn short_hash<'a>(hash: &'a str) -> &'a str
```

Read `'a` as a lifetime name (pronounced "tick-a"). The signature says: "the returned `&str` lives for the same span `'a` as the input `&str`." So if you tried to use the result after `hash` was dropped, the borrow checker would reject it. You didn't have to write `'a` - the compiler applied **lifetime elision**, a set of rules for the common cases: one input reference means the output borrows from it.

Methods that return borrows of `self`

The other elision rule: if a method takes `&self`, any returned reference is assumed to borrow from `self`. `Source::as_file` is a clean example:

```rust
// crates/rustyweb-lib/src/collections.rs:38
pub fn as_file(&self) -> Option<&Path> {
    match self {
        Source::File(p) => Some(p.as_path()),
        Source::Url(_) => None,
```

```
    }
}
```

The returned `&Path` points *inside* the `Source` (specifically at the `PathBuf` held by the `File` variant). The full signature is `fn as_file<'a>(&'a self) -> Option<&'a Path>`: the borrowed path can't outlive the `Source` it came from. Try to keep the `&Path` around after the `Source` is dropped and you get a compile error - the reference is tied to the source's lifetime.

`'static` - "lives for the whole program"

There's one special lifetime with a name you *do* write: `'static`, meaning "this reference is valid for the entire duration of the program." String literals are `'static` because they're baked into the compiled binary. See:

```rust
// crates/rustyweb-lib/src/server.rs:718
fn mime_guess_from_path(path: &str) -> &'static str {
    if path.ends_with(".html") {
        "text/html; charset=utf-8"
    } else if path.ends_with(".js") || path.ends_with(".mjs") {
        "application/javascript"
    } ...
    } else {
        "application/octet-stream"
    }
}
```

This is a great contrast to `short_hash`. There, the output borrowed *from the input*. Here, the input `path` is a borrowed `&str` of some unknown lifetime, but the output is `&'static str` - the returned string does **not** borrow from `path` at all. It's one of a handful of string literals that live forever. So the caller can hold onto the result as long as it likes, regardless of what happens to `path`. The lifetimes in the signature document that independence precisely.

You'll also spot `'static` as a *bound* in the logging setup (`main.rs:25`, `N: ... + 'static`), where it means "this type contains no short-lived borrowed references" - a requirement we'll see again with async.

The anonymous lifetime `<'_>`

Throughout the code you'll see `<'_>`:

```rust
// crates/rustyweb-lib/src/collections.rs:86
impl std::fmt::Display for Source {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        f.write_str(&self.location())
    }
}
```

`Formatter<'_>` means "a `Formatter` that holds a reference with *some* lifetime - compiler, please infer it, I don't need to name it." The `<'_>` is a deliberate signal to the reader that a lifetime exists here (the `Formatter` borrows the underlying output buffer) without cluttering the code with a name. It's the lifetime equivalent of the `_` you've seen in patterns.

The insight: this codebase mostly *avoids* lifetimes on purpose

Here's something worth noticing. Look at the long-lived data types in rustyweb - `WarcRecord` (`warc.rs:6`), `Collection` (`collections.rs:92`), `AppState` (`server.rs:28`). **None of them has a lifetime parameter.** They store `String`, `PathBuf`, `Vec<u8>` - *owned* data - never `&str`, `&Path`, or `&[u8]`.

That's a deliberate design choice, and a common one in real Rust. You *can* write a struct that borrows:

```
// hypothetical - NOT in this codebase
struct WarcRecord<'a> {
    target_uri: &'a str,    // borrows from a buffer owned elsewhere
}
```

But then the struct carries a lifetime parameter `<'a>` that infects every function touching it, and the record can't outlive the buffer it points into. For data that's parsed once and then passed around, merged into hash maps, sent across threads, and stored in a manifest, that's a straitjacket. So rustyweb pays for owned `String`s (a heap allocation each) and buys freedom: the records are *self-contained* and lifetime-free.

The takeaway for a learner: **lifetimes appear mostly at function boundaries that return borrows, not on your data structures.** When you own your data, the borrow checker mostly gets out of your way. Reaching for a lifetime parameter on a struct is a real technique, but it's an optimization you adopt deliberately, not a default. rustyweb shows the pragmatic default.

Advanced sighting: higher-ranked trait bounds

One genuinely advanced piece of lifetime syntax appears in `main.rs`, in the custom log formatter's `where` clause:

```
// crates/rustyweb-bin/src/main.rs:22
impl<S, N> FormatEvent<S, N> for ColorLineFormat
where
    S: tracing::Subscriber + for<'a> LookupSpan<'a>,
    N: for<'a> tracing_subscriber::fmt::FormatFields<'a> + 'static,
{ ... }
```

`for<'a> LookupSpan<'a>` is a **higher-ranked trait bound** (HRTB). It reads: "for *any* lifetime `'a`, this type implements `LookupSpan<'a>`." You reach for this when a trait is itself parameterized by a lifetime and you need it to hold for all

possible lifetimes, not one specific one. You don't need to write these often - they show up mostly when plugging into generic frameworks like `tracing` - but now you know what `for<'a>` means when you see it.

---

13. Async, in depth

Part I introduced `async fn` and `.await` (§7c). Here's what's really happening, because async has a few surprising properties.

Futures are inert - nothing runs until polled

In most languages, calling an async function *starts* it (JavaScript promises are "eager" - they begin executing immediately). In Rust, calling an `async fn` returns a `Future`, which is an inert value describing work to be done. It does **nothing** until something *drives* it - either you `.await` it, or you hand it to the runtime.

This is visible in the shutdown code, where two futures are *defined* but not yet run:

```
// crates/rustyweb-bin/src/main.rs:148
let ctrl_c = async {
    tokio::signal::ctrl_c()
        .await
        .expect("failed to listen for ctrl-c");
    tracing::info!("received shutdown signal");
};
```

That `async { ... }` block creates a future and binds it to `ctrl_c`. At this point *no signal handler is listening yet* - the code inside hasn't executed. It only runs when it's polled, which happens here:

```
// crates/rustyweb-bin/src/main.rs:167
tokio::select! {
    result = rustyweb_lib::server::serve(&bind, &home) => { result?; }
    _ = ctrl_c => {}
    _ = terminate => {}
}
```

`tokio::select!` polls all three futures concurrently and runs the arm of **whichever finishes first**, then drops the others. So: the server runs; the moment Ctrl-C *or* SIGTERM fires, its future completes, `select!` returns, the server future is dropped (shutting it down), and `main` returns. That's the whole graceful-shutdown mechanism, and it works precisely *because* futures are values you can hold, race, and drop.

The runtime and `#[tokio::main]`

A future needs an **executor** (a runtime) to be polled to completion. Bare `main` can't be `async` - something has to bootstrap the async world from the synchronous one. That's what the attribute does:

```rust
// crates/rustyweb-bin/src/main.rs:105
#[tokio::main]
async fn main() -> Result<()> { ... }
```

`#[tokio::main]` is a macro that rewrites your `async fn main` into roughly:

```rust
fn main() -> Result<()> {
    tokio::runtime::Runtime::new().unwrap().block_on(async {
        // ... your original body ...
    })
}
```

So there's an ordinary synchronous `main` underneath that builds a runtime and calls `block_on` - the one place the sync world blocks to drive the async world to completion. Everything with `.await` runs inside that.

`.await` is a suspension point

When execution reaches `.await`, the future may not be ready (e.g. the TCP socket has no bytes yet). Instead of blocking the thread, the task **suspends** and hands the thread back to the runtime, which runs *other* ready tasks. When the awaited operation becomes ready, the runtime resumes the task where it left off.

This is why one thread can serve thousands of connections: while connection A waits on `.await` for disk or network, the thread is off serving connections B, C, D. Between `.await` points, code runs normally and synchronously.

```rust
// crates/rustyweb-lib/src/server.rs:94
pub async fn serve(bind: &str, home: &Path) -> Result<()> {
    let app = router(home)?;                                    // sync
    let listener = tokio::net::TcpListener::bind(bind).await?;  // may suspend
    tracing::info!("listening on {bind}");                      // sync
    axum::serve(listener, ...).await?;                          // suspends, ~forever
    Ok(())
}
```

Note `.await?` stacks the two ideas from Part I: `.await` gets the `Result` out of the future, then `?` propagates it if it's an error.

The connection to lifetimes: why `Arc`, not `&`

Here's where §12 and §13 meet, and it's the single most important thing to understand about async Rust.

A suspended task can be **resumed later, possibly on a different thread**, and it can outlive the function that spawned it. So the compiler requires that data living across `.await` points and shared between tasks be `'static` (no short-lived borrows) and `Send` (safe to move between threads).

That means you generally **cannot** hand an async handler a plain reference `&AppState` - the borrow checker can't prove the reference will outlive the task, because in general it won't. The solution is shared *ownership* instead of borrowing:

```
// crates/rustyweb-lib/src/server.rs:43
let state = Arc::new(AppState { search, home: ..., index_dir: ... });

// crates/rustyweb-lib/src/server.rs:108
async fn homepage(State(state): State<Arc<AppState>>) -> impl IntoResponse { ... }
```

`Arc<AppState>` is an **A**tomically **R**eference-**C**ounted pointer: it's owned, `'static`, and cloning it just bumps a counter and hands out another handle to the *same* `AppState`. Every request handler gets its own `Arc` clone; the `AppState` itself is freed only when the last handler holding a clone is done. This is the async-world answer to "how do many tasks share one thing without borrowing" - and it's *why* `AppState` (§12) stores owned `PathBuf`s rather than borrowed `&Path`s. If it held borrows, it couldn't be `'static`, and it couldn't go in an `Arc` shared across tasks. The ownership design and the async design are the same decision.

`async fn` returns `impl Future`

Tying back to `impl Trait` (§6): an `async fn` is sugar. This…

```
pub async fn serve(bind: &str, home: &Path) -> Result<()> { ... }
```

…is essentially…

```
pub fn serve(bind: &str, home: &Path) -> impl Future<Output = Result<()>> { ... }
```

The function returns "some future that, when driven, yields a `Result<()>`." The `async` keyword just writes that future for you from ordinary-looking code.

Streaming: async that never buffers the whole file

A subtle, real payoff of async I/O is in file serving. A WACZ can be gigabytes; loading it into memory to answer a range request would be a disaster. Instead the server turns the file into an async *stream* of chunks:

```
// crates/rustyweb-lib/src/server.rs:576
let length = end - start + 1;
let limited = tokio::io::AsyncReadExt::take(file, length);    // async reader, capped at `le
let body = Body::from_stream(ReaderStream::new(limited));     // reader -> stream of chunks
```

`ReaderStream` adapts an async reader into a `Stream` of byte chunks, and `Body::from_stream` makes that the HTTP response body. Bytes flow from disk to socket a chunk at a time, the task suspending at each `.await` when the socket isn't ready - constant memory, no matter the file size. The `.take(length)` for range requests is the async cousin of the iterator `.take(14)` you saw in §4: same idea, "stop after N," applied to an async byte reader.

Two kinds of concurrency, both in one program

It's worth stepping back: rustyweb uses **two completely different concurrency tools for two different problems**, which is a great illustration of how Rust thinks about this.

- **Indexing is CPU-bound** (parsing, decompressing, text extraction). The bottleneck is compute, so it uses **Rayon** (§7a) to spread work across *all CPU cores* with `.par_iter()`.
- **Serving is I/O-bound** (waiting on sockets and disk). The bottleneck is waiting, so it uses **Tokio async** to juggle *many connections on few threads* by suspending at `.await`.

Threads-for-compute, async-for-waiting. Using async for the CPU-bound indexing, or Rayon for the I/O-bound server, would both be the wrong tool. Seeing both in one codebase is a good way to internalize the distinction.

---

14. Other things that are distinctly Rust

A tour of the smaller Rust-specific features woven through rustyweb that don't get their own section but are worth recognizing.

Tuple structs and the newtype pattern

The custom log formatter is a **tuple struct** - a struct whose fields are positional (accessed by `.0`, `.1`, …) rather than named:

```
// crates/rustyweb-bin/src/main.rs:14
struct ColorLineFormat(Format);
```

`ColorLineFormat` wraps a single `Format` value. You reach the inner value with `.0`:

```
// crates/rustyweb-bin/src/main.rs:49
self.0.format_event(ctx, Writer::new(&mut buf), event)?;
```

Wrapping one type in a single-field tuple struct is called the **newtype pattern**, and it's everywhere in idiomatic Rust. Here it exists so the code can attach *its own* behavior (colored output) on top of the library's default `Format`, by implementing the `FormatEvent` trait for the wrapper. `Source` and `Collection`

could also be seen through this lens - giving a plain value a distinct type so you can hang methods and trait impls on it.

Macros - the `!` means "this is a macro"

Every call with a `!` is a **macro**, not a function: `format!`, `vec!`, `println!`, `serde_json::json!`, `tracing::info!`, `tokio::select!`, `anyhow!`. Macros run at compile time and can do things functions cannot - take a variable number of arguments, accept custom syntax, and generate code.

- `format!("...")` and friends do type-checked string interpolation, and support **capturing variables by name from the surrounding scope**:

  ```
  // crates/rustyweb-bin/src/main.rs:51
  write!(writer, "{start}{line}\x1b[0m\n")
  ```

  `{start}` and `{line}` pull those local variables straight into the string - there's no positional argument list. (This inline-capture form is a relatively recent Rust feature; you'll also see the older `format!("{}", x)` style.)

- `serde_json::json!({ ... })` lets you write JSON-shaped literals directly in Rust and get a `serde_json::Value`:

  ```
  // crates/rustyweb-lib/src/server.rs:639
  let body = serde_json::json!({
      "results": results.iter().map(|r| serde_json::json!({
          "url": r.url,
          "title": r.title,
          ...
      })).collect::<Vec<_>>()
  });
  ```

- `tokio::select!` (§13) has entirely custom syntax - `pattern = future => body` arms - that no function could express.

There are two flavors under the hood: **declarative** macros (`macro_rules!`, like `vec!`) and **procedural** macros, which include the `#[derive(...)]` attributes from Part I. You mostly *use* macros rather than write them, but recognizing the `!` tells you "this has compile-time superpowers."

The turbofish `::<>`

When the compiler can't infer a type, you spell it out with the "turbofish":

```
// crates/rustyweb-lib/src/index.rs:239
.collect::<Result<Vec<_>>>()
```

```
// crates/rustyweb-lib/src/wacz.rs:65
serde_json::from_str::<DataPackage>(&buf)
```

`.collect()` can build many different collections, so `::<Result<Vec<_>>>` tells it which. `from_str` can deserialize into many types, so `::<DataPackage>` says which. The `_` inside (`Vec<_>`) means "infer *this* part" - so you only pin down what's ambiguous. The name comes from its shape: `::<...>` looks like a fish. You can often avoid it by annotating the variable's type instead (`let x: Vec<..> = ...`), and the codebase does both.

Slices - borrowed views into contiguous data

A **slice** is a reference to a *contiguous run* of elements - a pointer plus a length, borrowing data owned elsewhere, with no copying. `&str` is a slice of a `String`; `&[u8]` is a slice of bytes. You've seen them as function arguments (`fn extract_pdf_text(bytes: &[u8])`) and as sub-ranges:

```
// crates/rustyweb-lib/src/collections.rs:180
hasher.update(&buf[..n]);
```

`&buf[..n]` is "a view of the first `n` bytes of `buf`" - no allocation, just a window. This is how the SHA-256 loop feeds only the bytes it actually read into the hasher. Slices are central to Rust's zero-copy style: you pass views around instead of copying buffers. (`short_hash`'s `hash.get(..8)` in §12 is the same idea, returning a slice of the first 8 chars.)

Conditional compilation with `#[cfg(...)]`

Attributes starting with `cfg` include or exclude code *at compile time* based on configuration. Two uses appear in rustyweb.

Test code is compiled only during `cargo test`, never into the shipped binary:

```
// e.g. crates/rustyweb-lib/src/collections.rs:196
#[cfg(test)]
mod tests { ... }
```

And platform-specific code is selected per target OS - the SIGTERM handler exists only on Unix, with a do-nothing fallback elsewhere:

```
// crates/rustyweb-bin/src/main.rs:155
#[cfg(unix)]
let terminate = async { ... install SIGTERM handler ... };

#[cfg(not(unix))]
let terminate = std::future::pending::<()>();   // a future that never completes
```

On Windows, `terminate` becomes a future that never fires, so `select!` simply never takes that arm. The compiler picks exactly one definition; the other doesn't exist in the build. This is compile-time polymorphism over the environment, with no runtime cost.

The marker traits `Send` and `Sync`

`Send` ("safe to move to another thread") and `Sync` ("safe to share between threads by reference") are **marker traits** - they have no methods; they're just compiler-checked labels, and they're auto-derived for a type when all its fields qualify. You rarely write them, but they're the invisible machinery behind "fearless concurrency": the Rayon parallelism in §7a and the shared `Arc<AppState>` in §13 compile *only because* the compiler can prove the data crossing thread boundaries is `Send`/`Sync`. If you ever try to share something thread-unsafe, the error will mention these traits - now you know they're the guardrail, not a bug in your code.

Zero-cost abstractions

A recurring Rust promise: high-level constructs compile down to code as efficient as the hand-written low-level version. The iterator chain in `iso_to_14digit` (§4)…

```
s.chars().filter(|c| c.is_ascii_digit()).take(14).collect()
```

…compiles to essentially the same machine code as a hand-written `for` loop with an index and a counter - no intermediate collections, no per-element function-call overhead. Likewise `impl Trait` returns and generics are resolved at compile time (monomorphized) rather than through runtime indirection. The guiding idea: you don't pay a runtime tax for writing at a higher level. This is why the codebase can lean on expressive iterator pipelines without a performance guilt.

Closures and the three `Fn` traits

Closures - anonymous functions that can capture variables from their surrounding scope - are used constantly: `.map(|c| ...)`, `.find(|c| c.id == id)`, `.with_context(|| format!(...))`. The `|args| body` syntax defines one; an empty `||` takes no arguments.

Two things worth knowing. First, closures *capture their environment* - `.find(|c| c.id == id)` reaches out and uses the local `id`. Second, the laziness matters for correctness and performance: `.with_context(|| format!(...))` passes a closure, so the (potentially expensive) error message is built **only if there's actually an error** - on the happy path it's never called. Rust classifies closures into three traits (`Fn`, `FnMut`, `FnOnce`) by how they use what they capture (read, mutate, or consume), which is how the compiler knows whether a closure can be called repeatedly or moved across threads - but for reading this codebase, "anonymous function that can see nearby variables" is the mental model you need.

RAII and `Drop` - cleanup is a type's responsibility

Reinforcing §5 because it's so distinctly Rust: there is no `finally`, no `try-with-resources`, no garbage collector deciding *when* things get cleaned

up. Instead, a type can implement the `Drop` trait, and its cleanup code runs *automatically and deterministically* the instant the value goes out of scope. You've seen three payoffs of this one mechanism:

- `NamedTempFile` deletes itself when dropped, which is why `index_one` binds it to `_tmp` to control *when* that happens (§5, `index.rs:103`).
- A `MutexGuard` releases the lock when dropped, which is why the scoped `{ }` block in `index_wacz` releases it precisely (§4/§7b, `index.rs:276`).
- `gag::Gag` restores stdout when dropped, ended deliberately with `drop(quiet)` (§5, `main.rs:141`).

The pattern is called **RAII** (Resource Acquisition Is Initialization): acquiring a resource is tied to creating a value, and releasing it is tied to that value being dropped. In Rust, *ownership and cleanup are the same system* - which is why you almost never leak a file handle, lock, or temp file, and why there's no "did I remember to close this?" ritual.

---

16. Traits and generics, in depth

Part I introduced traits as "like interfaces" and showed `impl Trait` in return position. This section goes to the core of how Rust does abstraction - and a notable fact about rustyweb: it uses **only static dispatch**. There is not a single `dyn` or `Box<dyn Trait>` in the codebase. Understanding why is a good way to understand the whole system.

Generics - write once, specialize for many types

A **generic** function or type has a type *parameter* (conventionally `T`, or a more meaningful letter) that stands in for "some type the caller chooses." The WARC parser is generic over its reader:

```rust
// crates/rustyweb-lib/src/warc.rs:232
fn parse_one_warc_record<R: BufRead>(
    mut r: R,
    offset: u64,
    record_length: u64,
) -> Result<Option<WarcRecord>> { ... }
```

`<R: BufRead>` introduces a type parameter `R`, constrained to types that implement the `BufRead` trait. This one function works for *any* buffered reader - a file, an in-memory cursor, the counting wrapper - without being rewritten. In the plain-WARC path it's called with a `BufReader<Cursor<Vec<u8>>>` (`warc.rs:165`); in the gzip path with a cursor over decompressed bytes. Same code, different concrete `R` each time.

Trait bounds - the contract a generic relies on

`<R: BufRead>` is a **trait bound**. It's a promise in both directions: the caller must supply an `R` that implements `BufRead`, and in return the function body is allowed to call any `BufRead` method on `r` (like `read_line`). Without the bound, `R` could be *any* type and the compiler wouldn't let you call `.read_line()` on it - it can't assume methods that aren't guaranteed to exist.

Bounds can be written inline (`<R: BufRead>`) or, when they get long, in a `where` clause - which is exactly why the log formatter uses one:

```
// crates/rustyweb-bin/src/main.rs:22
impl<S, N> FormatEvent<S, N> for ColorLineFormat
where
    S: tracing::Subscriber + for<'a> LookupSpan<'a>,
    N: for<'a> tracing_subscriber::fmt::FormatFields<'a> + 'static,
{ ... }
```

`S: A + B` means "`S` must implement *both* `A` and `B`" - bounds compose with `+`. The `where` block is purely for readability; it means the same as cramming it all into the angle brackets.

Generic structs and generic impls

Types can be generic too. `CountingBufReader` wraps *any* reader:

```
// crates/rustyweb-lib/src/warc.rs:387
struct CountingBufReader<R: Read> {
    inner: BufReader<R>,
    count: u64,
}

impl<R: Read> Read for CountingBufReader<R> {
    fn read(&mut self, buf: &mut [u8]) -> std::io::Result<usize> {
        let n = self.inner.read(buf)?;
        self.count += n as u64;
        Ok(n)
    }
}
```

Read `impl<R: Read> Read for CountingBufReader<R>` carefully - the `<R: Read>` after `impl` *declares* the type parameter, and it appears again in the type being implemented. This says "for any readable `R`, the counting wrapper around it is itself readable." That's how the wrapper can slot in anywhere a reader is expected while transparently counting bytes.

Static vs dynamic dispatch (and why this codebase picks static)

Here's the deep part. There are two ways to write "code that works with any type implementing a trait":

- **Static dispatch** - generics (`<R: Read>`) and `impl Trait`. The compiler generates a *separate specialized copy* of the function for each concrete type used, a process called **monomorphization**. Calls are direct, can be inlined, and cost nothing at runtime. The trade-off is larger compiled code and that the type is fixed at each call site.
- **Dynamic dispatch** - `dyn Trait`, usually behind a pointer like `Box<dyn Trait>` or `&dyn Trait`. One copy of the code handles all types; the concrete type is erased and method calls go through a *vtable* (a pointer lookup) at runtime. Slightly slower, but lets you mix different concrete types in one collection (e.g. a `Vec<Box<dyn Trait>>`) and shrinks code size.

rustyweb chooses **static dispatch everywhere**. `iter_records` returns `impl Iterator<...>` rather than `Box<dyn Iterator<...>>`; the WARC parser is `<R: BufRead>` rather than `&mut dyn BufRead`. The payoff is the zero-cost story from §14 - the iterator chains and generic readers compile down to code as tight as hand-written loops. The codebase never needs the one thing dynamic dispatch buys you (a heterogeneous collection of differing types behind one trait), so it pays nothing for it. When you see `impl Trait` and `<T: Bound>` and *no* `dyn`, that's a deliberate "I want the fast, statically-known path" signal.

Associated types - traits with an "output type"

Some traits carry a type *inside* them, called an **associated type**. The two you meet immediately are `Iterator` and `Future`:

```
// crates/rustyweb-lib/src/warc.rs:28
pub fn iter_records(path: &Path) -> Result<impl Iterator<Item = Result<WarcRecord>>>
```

`Iterator` has an associated type `Item` - "what this iterator yields." Writing `impl Iterator<Item = Result<WarcRecord>>` pins that associated type: this iterator yields `Result<WarcRecord>` values. Similarly, `Future` has an associated type `Output` (§13): an `async fn serve(...) -> Result<()>` produces an `impl Future<Output = Result<()>>`. Associated types differ from generic parameters in that the *implementing type* chooses them, not the caller - a given iterator has exactly one `Item` type, decided by how it's built.

Extension traits: methods live in traits you must bring into scope

This one surprises newcomers and appears several times in rustyweb. In Rust, a method is only callable if the trait that defines it is **in scope** (imported). So you sometimes `use` a trait purely to unlock methods, even though you never name the trait again. Three real examples:

```rust
// crates/rustyweb-lib/src/collections.rs:169
use sha2::Digest;                    // brings .update() and .finalize() into scope
...
let mut hasher = sha2::Sha256::new();
hasher.update(&buf[..n]);            // <-- only works because Digest is imported

// crates/rustyweb-lib/src/index.rs:6
use rayon::prelude::*;               // brings .par_iter() onto standard collections

// crates/rustyweb-lib/src/server.rs:572
use tokio::io::AsyncSeekExt;         // brings async .seek() onto the file
if let Err(e) = file.seek(std::io::SeekFrom::Start(start)).await { ... }
```

`rayon::prelude::*` is how `.par_iter()` (§7a) appears on an ordinary `Vec` - it's not an inherent method, it's added by an extension trait. `AsyncSeekExt` and `AsyncReadExt` (the `Ext` suffix is a naming convention for exactly this) add async I/O methods. `Digest` adds the hashing methods. If you ever see an error like "method `update` not found" when you're *sure* it exists, the usual fix is importing the trait - and now you know why.

(A related detail: at `server.rs:577`, `tokio::io::AsyncReadExt::take(file, length)` calls the trait method by its *fully-qualified path* instead of importing the trait - another way to reach a trait method, useful when two traits would otherwise both offer a `take`.)

The `From` trait is secretly what makes `?` work

You met `From` in Part I as the conversion behind serde's string representation of `Source`. It has a second, hidden role: **the `?` operator uses `From` to convert error types.** When you write `something()?` and the error type coming out (`io::Error`, say) differs from the function's declared error type (`anyhow::Error`), `?` automatically calls `From::from` to convert it. That's the entire trick behind why one `anyhow`-returning function can `?` a file error, a JSON error, and a network error all in the same body (§3): `anyhow::Error` implements `From<E>` for essentially every standard error type, so `?` converts each on the way out. Traits aren't just interfaces here - they're wired into the language's control-flow operators.

`Default` - the "empty value" trait

`Default` provides `T::default()`, a canonical zero/empty value, and it's usually derived. rustyweb leans on it for best-effort parsing:

```rust
// crates/rustyweb-lib/src/index.rs:218
#[derive(Default)]
struct MergedPage {
    timestamp: String,
    title: Option<String>,
    html_body: Option<String>,
```

```
    rendered_text: Option<String>,
}
```

Deriving `Default` gives a `MergedPage` with an empty string and three `None`s. That powers two idioms you've seen:

- `pages.entry(url).or_default()` (`index.rs:256`) - "get the entry for this URL, or insert a fresh default `MergedPage` if absent," the standard way to accumulate into a `HashMap`.
- `.unwrap_or_default()` (`index.rs:117`, `wacz.rs:66`) - "use the value, or a default if it's `None`/`Err`," which is how metadata reading degrades gracefully when a WACZ is missing fields.

The orphan rule - why the newtype pattern exists

One coherence rule explains a pattern from §14. Rust says you can implement a trait for a type only if **you own the trait or you own the type** (the "orphan rule"), so that two crates can't define conflicting impls. That's a problem when you want to customize a *foreign* trait on a *foreign* type - which is exactly the log formatter's situation: both `FormatEvent` (from `tracing-subscriber`) and `Format` (also from `tracing-subscriber`) are foreign. The fix is to wrap the foreign type in a *local* newtype and implement the foreign trait on that:

```
// crates/rustyweb-bin/src/main.rs:14
struct ColorLineFormat(Format);            // local wrapper around a foreign type
// crates/rustyweb-bin/src/main.rs:22
impl<S, N> FormatEvent<S, N> for ColorLineFormat { ... }   // now allowed
```

Because `ColorLineFormat` is defined in this crate, implementing a foreign trait on it is legal. The newtype pattern (§14) isn't just for adding meaning - it's the standard escape hatch around the orphan rule.

---

17. Serde derives, in depth

`serde` (**ser**ialize/**de**serialize) is the backbone of every JSON boundary in rustyweb - the manifest, WACZ metadata, CDX records, the search API. Part I said "the struct definition *is* the schema." Here's how that actually works.

What `#[derive(Serialize, Deserialize)]` generates

`serde` splits into two halves: a small core crate defining the `Serialize` and `Deserialize` traits and an abstract "data model," and format crates like `serde_json` that map that model to a concrete syntax. When you write:

```
// crates/rustyweb-lib/src/collections.rs:6
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
pub struct SeedPage {
```

```
    pub url: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    pub title: Option<String>,
    pub ts: String,
}
```

the `Serialize`/`Deserialize` derives are **procedural macros**: at compile time they read the struct's fields and *generate* the code that walks a `SeedPage` field by field (for serializing) or builds one up from incoming data (for deserializing). You never write that code, and it's specialized to this exact struct - no runtime reflection, no schema lookups. Then `serde_json::to_string` plugs the JSON syntax into the generated walk. The struct is the single source of truth: change a field and both directions update automatically.

`Option<T>` fields = optional keys

The most important everyday rule: a field of type `Option<T>` is an *optional* key. On deserialize, a missing key becomes `None` rather than an error; a present key becomes `Some(value)`. This is why the WACZ metadata parser can tolerate wildly varying files:

```
// crates/rustyweb-lib/src/wacz.rs:43
#[derive(Deserialize, Default)]
struct Metadata {
    title: Option<String>,
    description: Option<String>,
    created: Option<String>,
    mtime: Option<i64>,
}
```

A WACZ with none of these keys still deserializes fine - you just get four `None`s. The type declares "these may or may not be here," and serde honors it.

Field attributes fine-tune the mapping

The `#[serde(...)]` attributes are how you adjust the generated code without hand-writing it. Three appear in rustyweb, all on `Collection`:

```
// crates/rustyweb-lib/src/collections.rs:92
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Collection {
    pub id: String,
    #[serde(alias = "path")]
    pub source: Source,
    ...
    #[serde(skip_serializing_if = "Option::is_none")]
    pub description: Option<String>,
    #[serde(default, skip_serializing_if = "Vec::is_empty")]
```

```
    pub seed_pages: Vec<SeedPage>,
}
```

- **`alias = "path"`** - when *reading*, accept the key `path` as a synonym for `source`. This is the backward-compatibility hook: older manifests wrote `path`, newer ones write `source`, and both deserialize into the same field. There's a test proving it (`collections.rs:239`, `manifest_reads_legacy_path_key`).
- **`skip_serializing_if = "Option::is_none"`** - when *writing*, omit this key entirely if the value is `None`. Keeps the JSON tidy: a collection with no description simply has no `description` key rather than `"description": null`.
- **`default`** on `seed_pages` - when *reading*, if the key is absent, use `Vec::default()` (an empty vec) instead of failing. Combined with `skip_serializing_if = "Vec::is_empty"`, an empty seed list round-trips as "no key at all." (`#[serde(default)]` is what lets a *non-`Option`* field be optional on input.)

Container attributes: making `Source` serialize as a plain string

This is the most elegant serde usage in the codebase, and it ties directly back to the `From` trait (§16). Left alone, serde serializes an enum with **external tagging** - `Source::File(path)` would become `{"File": "archive/x.wacz"}`. That's ugly in a hand-editable manifest. So the enum carries a *container* attribute:

```rust
// crates/rustyweb-lib/src/collections.rs:16
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
#[serde(from = "String", into = "String")]
pub enum Source {
    File(PathBuf),
    Url(String),
}
```

`#[serde(into = "String")]` tells serde: "to serialize a `Source`, first convert it into a `String` (using the `From<Source> for String` impl), then serialize *that*." `#[serde(from = "String")]` says the reverse: "to deserialize, read a `String`, then run `From<String> for Source` to reconstruct the enum." Those two `From` impls (`collections.rs:74-84`) - one calling `.location()`, one calling `Source::parse` - are the entire bridge. The result: a `Source` reads and writes as a bare JSON string (`"archive/x.wacz"` or `"https://..."`), and the file/URL distinction is recovered by inspecting the string on the way back in. The test `source_serializes_as_plain_string` (`collections.rs:228`) locks this behavior in. This is a clean demonstration of composing two features - derive plus a manual `From` conversion - to get exactly the wire format you want.

Local throwaway structs as parsers

You don't need a top-level type for every JSON shape. serde structs are cheap, so rustyweb defines them *inside functions*, scoped to a single parse:

```rust
// crates/rustyweb-lib/src/wacz.rs:87
#[derive(Deserialize)]
struct PageEntry {
    url: Option<String>,
    title: Option<String>,
    ts: Option<String>,
}
if let Ok(p) = serde_json::from_str::<PageEntry>(line) {
    if let Some(url) = p.url {
        meta.seed_pages.push(SeedPage { url, title: p.title, ts: p.ts.unwrap_or_default() })
    }
}
```

`PageEntry` exists only to describe one line of `pages.jsonl`. It's declared right where it's used, keeps the module namespace clean, and documents the exact shape being parsed. The same technique appears with `DataPackage`/`Metadata` (`wacz.rs:43`) and `CdxJson` (`wacz.rs:184`). This "define a struct to match the JSON, then `from_str` into it" flow is the serde workhorse.

`serde_json::Value` for genuinely messy data

Sometimes the incoming JSON *isn't* consistently typed, and a fixed field type would reject valid data. WACZ CDX files are the poster child - they store numbers sometimes as JSON numbers, sometimes as quoted strings. The fix is to deserialize those fields as `serde_json::Value` ("any JSON value, I'll sort it out later") and coerce manually:

```rust
// crates/rustyweb-lib/src/wacz.rs:184
#[derive(Deserialize)]
struct CdxJson {
    url: Option<String>,
    status: Option<serde_json::Value>,   // number OR string in the wild
    offset: Option<serde_json::Value>,
    length: Option<serde_json::Value>,
    ...
}

// crates/rustyweb-lib/src/wacz.rs:195
fn coerce_u64(v: &Option<serde_json::Value>) -> u64 {
    match v {
        Some(serde_json::Value::Number(n)) => n.as_u64().unwrap_or(0),
        Some(serde_json::Value::String(s)) => s.parse().unwrap_or(0),
        _ => 0,
```

```
    }
}
```

This is the escape valve from strict typing: `Value` is serde's dynamically-typed representation, and the `match` (§4) handles each shape. The comment at `wacz.rs:180` explains the motivation - one quoted number shouldn't cause the whole record to be dropped. Use strict typed structs by default; drop to `Value` only where the data forces you to.

The three entry points you'll actually call

Across the codebase, serde_json shows up as just three calls:

- **`serde_json::from_str::<T>(text)`** - parse text into a `T`. Returns a `Result`, so it's `?`-propagated (`collections.rs:131`) or handled with `if let Ok` when failure is tolerable (`wacz.rs:65`).
- **`serde_json::to_string_pretty(&value)`** - serialize to nicely-indented JSON, used when writing the manifest so it's human-readable (`collections.rs:150`).
- **`serde_json::json!({ ... })`** - the macro (§14) for building a `serde_json::Value` inline, used to assemble the search API response (`server.rs:639`) without defining a response struct.

That's the whole surface. Between typed structs for known shapes, `Value` for messy ones, and these three calls, serde covers every JSON boundary in rustyweb - and in almost any Rust program you'll write.

---

18. A closing mental model

If you internalize just a few connected ideas, the rest of Rust follows:

1. **Every value has one owner; cleanup happens when the owner is dropped** (RAII, §5, §14). No GC, no `finally` - the type system *is* the resource manager.
2. **You can borrow instead of own, and lifetimes prove borrows never dangle** (§12). Own your data in structs and lifetimes stay out of your way; they surface mainly when a function returns a borrow.
3. **Fallibility and absence are values, not control-flow surprises** - `Result` and `Option`, handled with `match`/`?`/`if let` (§2, §3, §4). No exceptions.
4. **Async makes tasks into values you hold, race, and drop** (§13), and its `'static`/`Send` requirements are *why* shared state is owned and `Arc`-wrapped rather than borrowed - the same ownership rules from idea 1, projected onto concurrency.
5. **Abstractions are zero-cost** (§14), so you're free to write expressively - enums, iterators, closures, generics - without a runtime penalty.

6. **Traits are the unit of abstraction, and generics specialize them for free** (§16). Behavior is shared by implementing traits, not by inheritance; generics plus `impl Trait` give you polymorphism resolved at compile time. Traits even power the language itself - `?` converts errors through `From`, and `serde` derives generate `Serialize`/`Deserialize` impls so your types *are* your JSON schema (§17).

Everything in rustyweb is an application of these. When a piece of code looks strange, ask which of the six it's serving - usually it's ownership (1/2) or error handling (3), and the "weird" syntax is just the compiler being made to prove something it would otherwise have to trust.